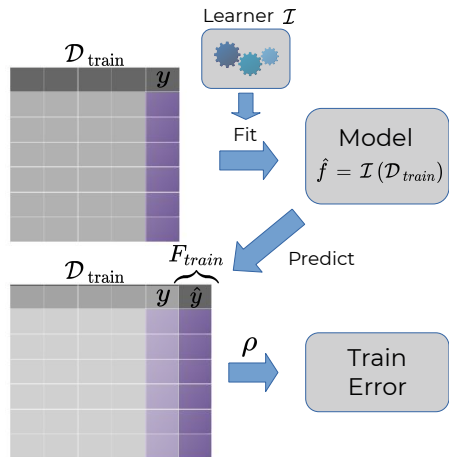


Outline

Training error

- ▶ Simplest evaluation: plug training data back into the trained model
- ▶ Aka **apparent error** or **resubstitution error**
- ▶ Often *very* optimistic — model has already seen these points
- ▶ 1-NN: training error is always 0 (each point is its own nearest neighbor)
- ▶ Any model storing training data + replaying labels achieves zero training error
- ▶ Interpolating splines / Gaussian processes can perfectly match noise too



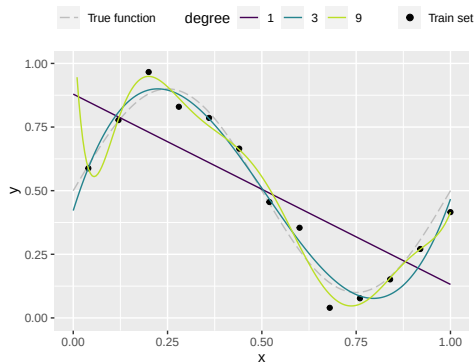
Why training error is unreliable

Polynomial regression on

$$y = 0.5 + 0.4 \sin(2\pi x) + \varepsilon:$$

- ▶ $d = 1$: train MSE = 0.036 (underfit)
- ▶ $d = 3$: train MSE = 0.003 (good)
- ▶ $d = 9$: train MSE = 0.001 (overfit)

Training error *monotonically decreases* with complexity. Used alone, it always picks the most complex model — **wrong**.



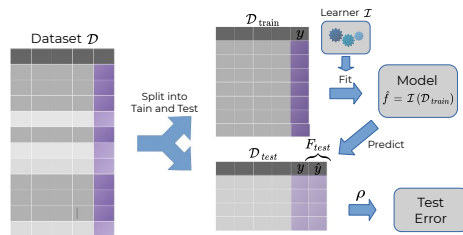
Classical GoF measures also use training data

- ▶ R^2 , log-likelihood, AIC, BIC, deviance — all computed on training data
- ▶ Work *sometimes*: restricted-capacity models, plenty of data, distributional assumptions hold
- ▶ Hard to gauge *when they break*, especially for high-dim or complex data
- ▶ No clean way to compare them across model families (e.g., LM vs random forest)

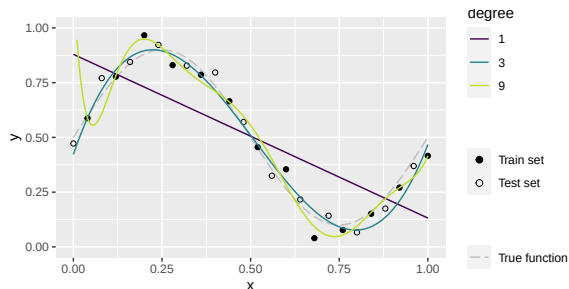
Out-of-sample testing is the safer default.

Hold-out splitting

- ▶ Simulate “new data” by partitioning $\mathcal{D}$ into $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$
- ▶ Train on $\mathcal{D}_{\text{train}}$, evaluate on $\mathcal{D}_{\text{test}}$
- ▶ Common split: 2/3 train, 1/3 test
- ▶ Aka the **untouched test set** principle — never train on, never tune on, never even peek at, the test set



Same polynomial example, now with test set



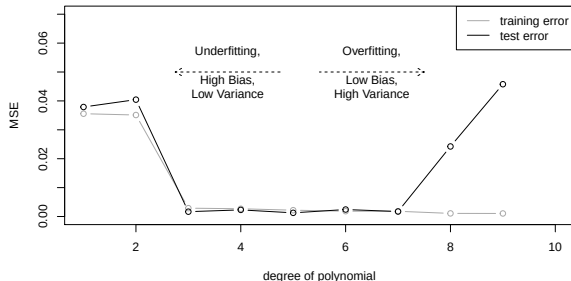
► $d = 1$: test MSE = 0.038 (underfit)

► $d = 3$: test MSE = 0.002 (good)

► $d = 9$: test MSE = **0.046** (overfit)

Test error *exposes* the overfitting that training error hid.

The U-shape of test error



Increasing model complexity:

- ▶ Training error: **monotonically decreases**
- ▶ Test error: **U-shape**
(underfit → sweet spot → overfit)

The U's minimum is the "sweet spot."
Finding it is what tuning (L05) is for.

Bias-variance tradeoff of hold-out itself

Choice of split ratio matters:

- ▶ **Small training set** $\rightarrow$ poor fit, *pessimistic* bias in estimated GE
- ▶ **Small test set** $\rightarrow$ high variance in the estimate (one bad test point swings the metric)
- ▶ Empirically: train/test $\approx 2/3$ ratio minimizes MSE of the holdout estimator

- ▶ Single split estimates fluctuate; *repeated* holdout or *cross-validation* reduce variance (L05)
- ▶ Small datasets: holdout is risky $\Rightarrow$ use CV

Generalization error

Our actual quantity of interest:

$$GE(f) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathbb{P}_{xy}} [L(y, f(\mathbf{x}))]$$

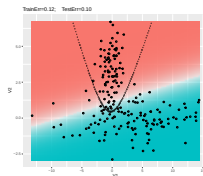
Average loss on a fresh sample from the true distribution.

- ▶ We never observe $GE(f)$ directly (we don't have $\mathbb{P}_{xy}$)
- ▶ Test error is our *estimator* of $GE(f)$
- ▶ As $|\mathcal{D}_{\text{test}}| \rightarrow \infty$, test error $\rightarrow GE(f)$ (assuming i.i.d.)
- ▶ Whole field of ML evaluation: build good estimators of $GE(f)$ from finite data

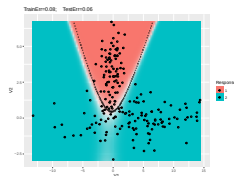
Underfitting vs. appropriate fit

Underfitting:

- ▶ Model can't capture the true relationship
- ▶ High train error *and* high test error
- ▶ Hard to detect — we don't know the Bayes error



underfit

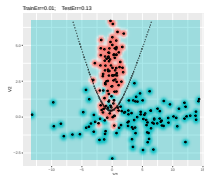


appropriate

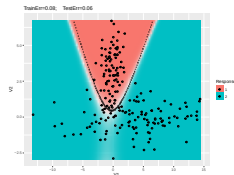
Overfitting vs. appropriate fit

Overfitting:

- ▶ Model fits noise / artifacts in training data
- ▶ *Small* train error, *large* test error
- ▶ Predictions on new data can't be trusted
- ▶ Proper evaluation workflow makes it visible

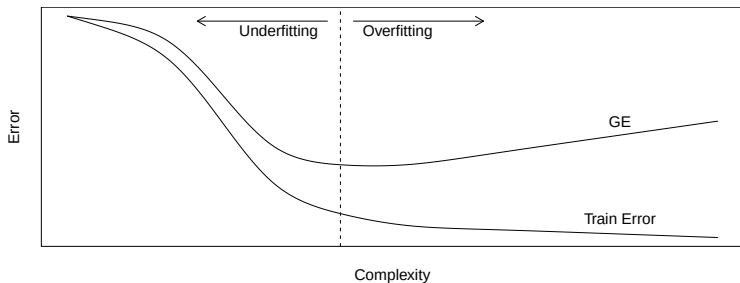


overfit



appropriate

Complexity vs. generalization error



- ▶ Common U-shape as complexity (or training rounds) grows
- ▶ Optimal complexity: simplest model whose GE isn't significantly worse than more complex alternatives (*Occam's razor*)

What drives overfitting

More potential for overfitting when:

- ▶ Hypothesis space $\mathcal{H}$ is complex
- ▶ Training data is small
- ▶ Feature space is high-dimensional
- ▶ Irreducible noise is large

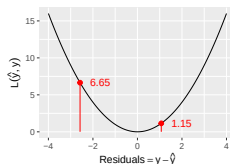
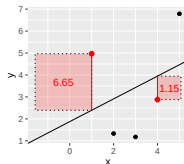
Implications:

- ▶ Bigger $\mathcal{H} \Rightarrow$ need more data to identify good models
- ▶ Less data $\Rightarrow$ stick with constrained $\mathcal{H}$
- ▶ Even linear models can overfit when p is comparable to or larger than n

Mean Squared Error (MSE)

$$\text{MSE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

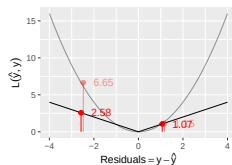
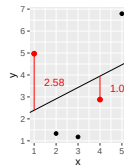
- ▶ $\rightarrow L_2$ loss averaged over the evaluation set
- ▶ **Outliers dominate** — squared residuals
- ▶ Variants: **SSE** (no division), **RMSE** (square root brings units back to y 's scale)



Mean Absolute Error (MAE)

$$\text{MAE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m |y^{(i)} - \hat{y}^{(i)}|$$

- ▶ $\rightarrow L_1$ loss averaged
- ▶ More **robust** to outliers
- ▶ Intuitive: “on average we’re off by X units”
- ▶ Variant: **Median absolute error** (even more robust)



MAPE — relative error

$$\text{MAPE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{m} \sum_{i=1}^m \left| \frac{y^{(i)} - \hat{y}^{(i)}}{y^{(i)}} \right|$$

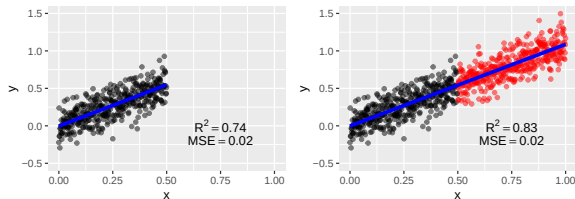
- ▶ Useful when targets span orders of magnitude (sales, rentals)
- ▶ Small $|y^{(i)}|$ dominates — undefined at $y^{(i)} = 0$
- ▶ Variants: MASE, sMAPE (symmetric)

R^2 : fraction of variance explained

$$R^2(\mathbf{y}, \hat{\mathbf{y}}) = 1 - \frac{\sum_i (y^{(i)} - \hat{y}^{(i)})^2}{\sum_i (y^{(i)} - \bar{y})^2} = 1 - \frac{\text{SSE}_{\text{model}}}{\text{SSE}_{\text{constant baseline}}}$$

- ▶ $R^2 = 1$: perfect predictions
- ▶ $R^2 = 0$: model no better than the constant $\bar{y}$
- ▶ $R^2 < 0$ (on test data!): *worse* than the constant — possible when $\bar{y}_{\text{train}} \neq \bar{y}_{\text{test}}$, or under overfitting
- ▶ Classic statistics-style measure. Often computed on training data, where it's $\in [0, 1]$ for any model better than the intercept.

R^2 vs MSE: don't be fooled



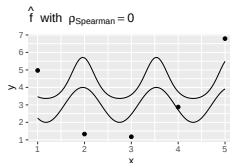
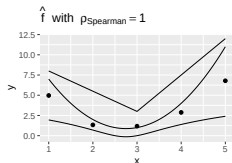
- ▶ Same DGP, fit on (a) half the data (black), (b) full data (black + red)
- ▶ Fit quality is essentially the same
- ▶ But R^2 goes *up* when you add data with more variance in x
- ▶ Because R^2 depends on the spread of y , not just the residuals

R^2 is comparable *within* a fixed dataset, not across.

Spearman's ρ : rank correlation

$$\rho_{\text{Spearman}}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{\text{cov}(\text{rg}(\mathbf{y}), \text{rg}(\hat{\mathbf{y}}))}{\sqrt{\text{var}(\text{rg}(\mathbf{y}))} \sqrt{\text{var}(\text{rg}(\hat{\mathbf{y}}))}} \in [-1, 1]$$

- ▶ Very robust to outliers
- ▶ Invariant to monotone transformations of $\hat{\mathbf{y}}$
- ▶ ± 1 : predictions and labels are in the same (or reverse) rank order
- ▶ Use when you care about *ranking* (recommender system, ordinal regression)



Choosing a metric

Quick decision guide:

- ▶ **MSE / RMSE**: default. Good when outliers are real signal.
- ▶ **MAE**: more robust; interpret in original units.
- ▶ **MAPE**: when relative error matters (sales, demand).
- ▶ R^2 : for communicating “variance explained” to non-technical audiences. Beware its caveats.
- ▶ **Spearman's ρ** : when only ranks matter.

Always evaluate on held-out / cross-validated data, not training data.

Avoiding overfitting (preview)

- ▶ Use more / better data (when feasible)
- ▶ Constrain $\mathcal{H}$: simpler model class
- ▶ Hyperparameter tuning to find the sweet spot (L05)
- ▶ Regularization (L03)
- ▶ Early stopping (L15)

Next: L05 — beyond hold-out. Cross-validation, hyperparameter tuning, the full evaluation toolkit.